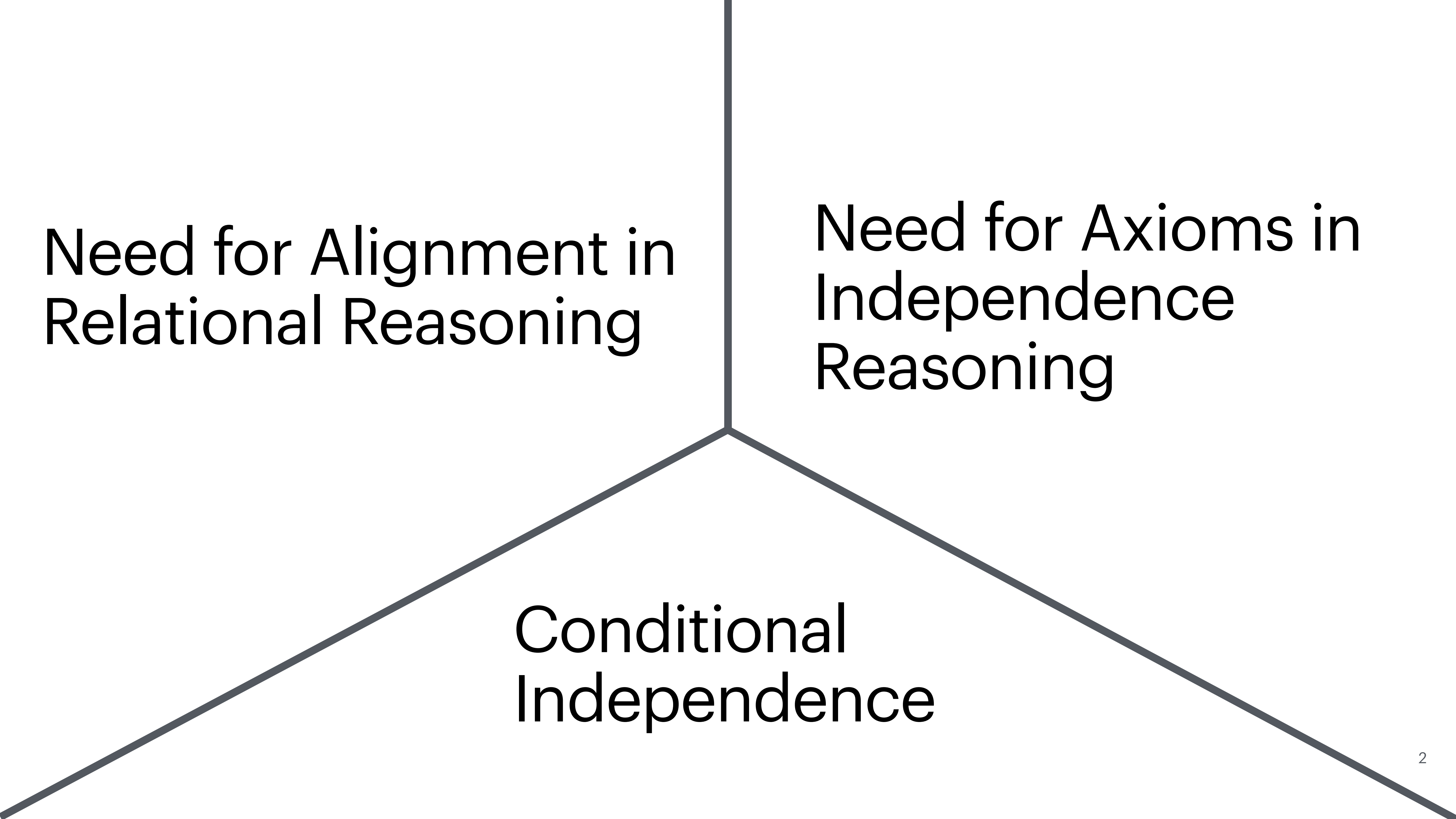


Bluebell: Relational Lifting Independence For Probabilistic Reasoning

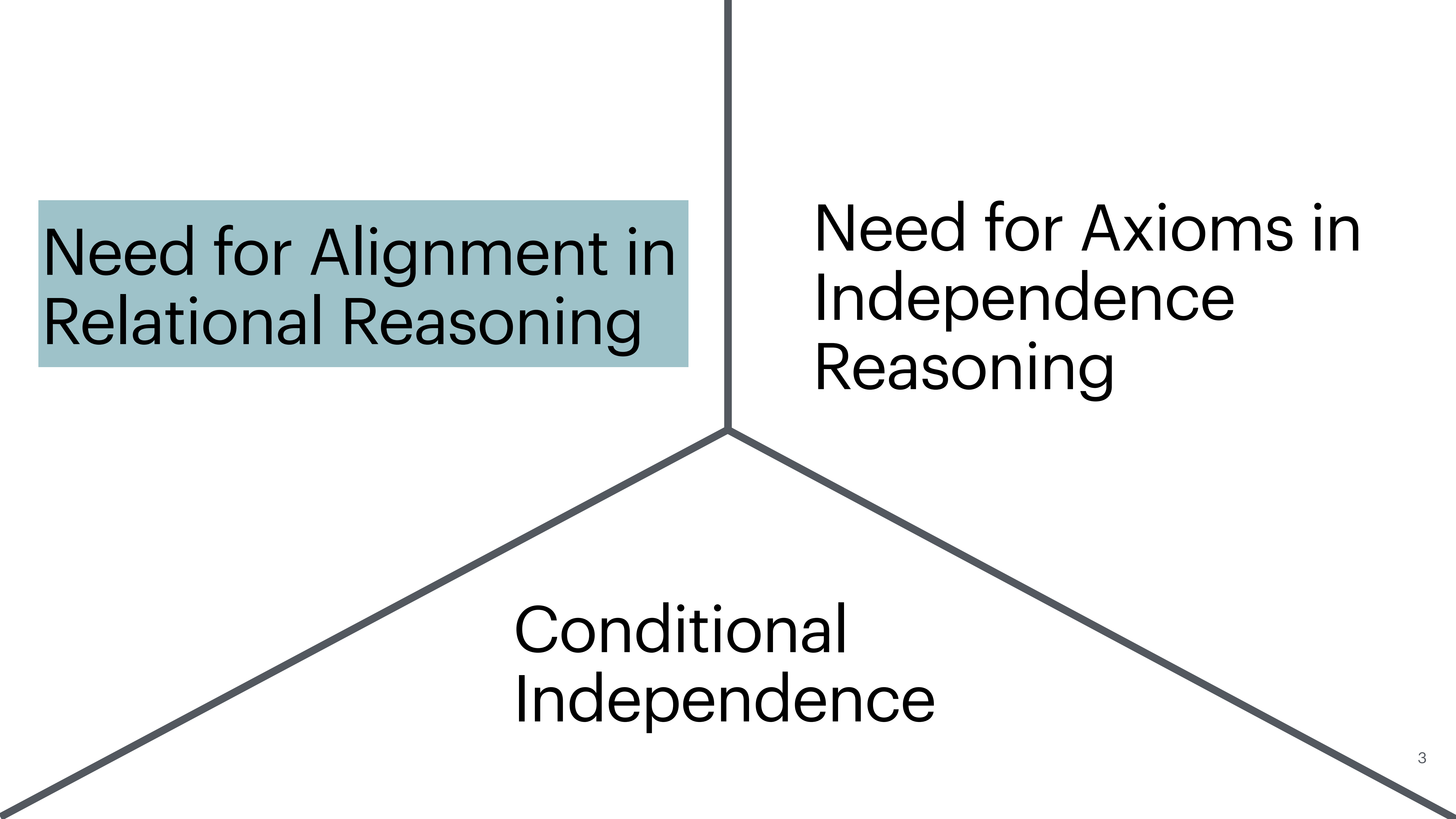
Joint work with Emanuele D'Ossualdo (U of Konstanz) and Azadeh Farzan (U of Toronto)



Need for Alignment in
Relational Reasoning

Need for Axioms in
Independence
Reasoning

Conditional
Independence

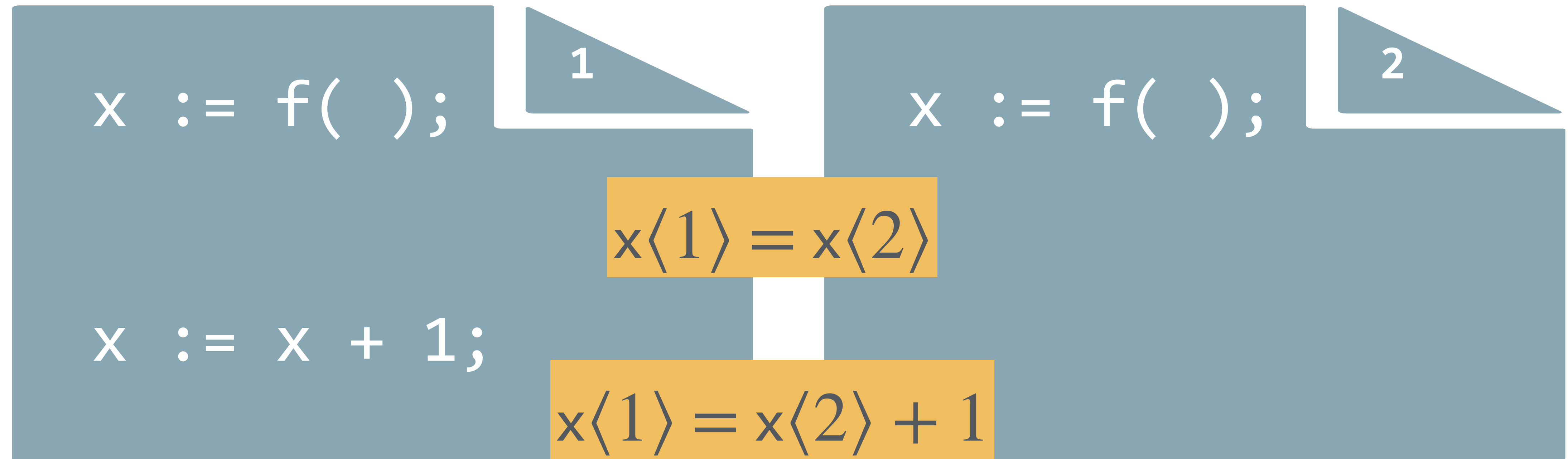


Need for Alignment in
Relational Reasoning

Need for Axioms in
Independence
Reasoning

Conditional
Independence

Relational Reasoning



handled by Relational Hoare Logic

Probabilistic Relational Reasoning

1

```
x :~ coin( );
```

```
x := x + 1;
```

2

```
x :~ coin( );
```

Probabilistic Coupling

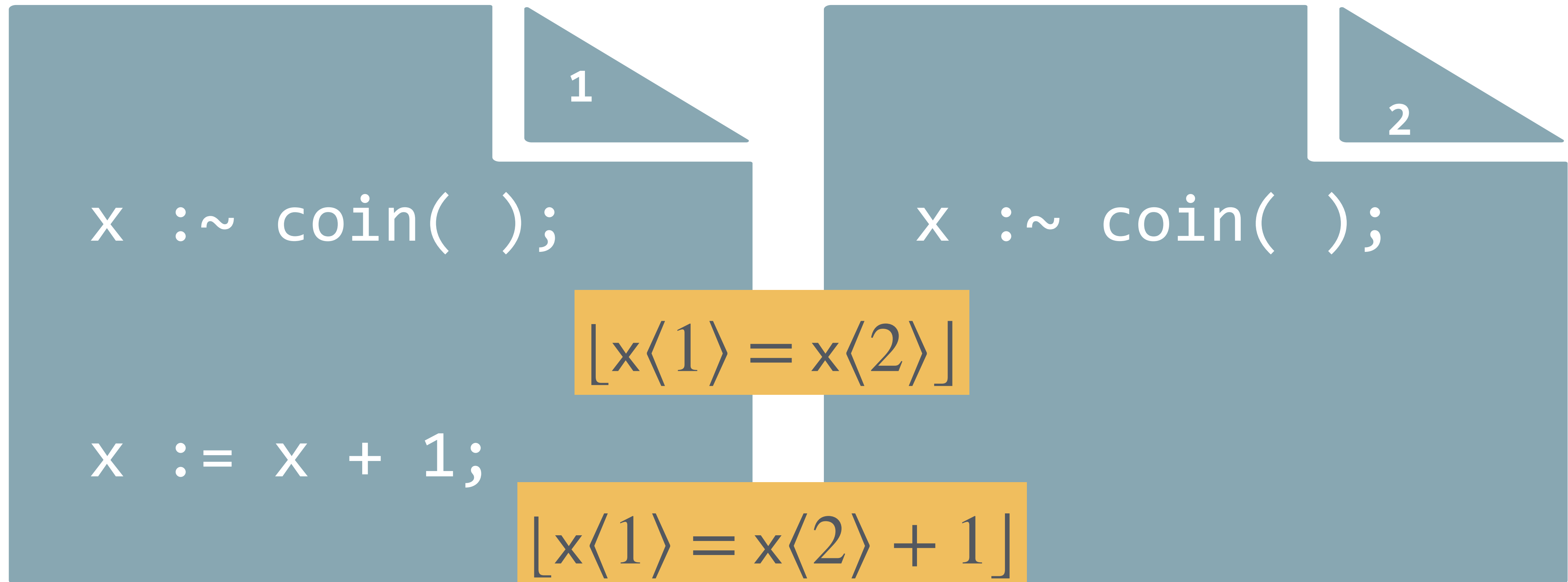
	$x_{\langle 1 \rangle} = H$	$x_{\langle 1 \rangle} = T$
$x_{\langle 2 \rangle} = H$	0.25	0.25
$x_{\langle 2 \rangle} = T$	0.25	0.25

	$x_{\langle 1 \rangle} = H$	$x_{\langle 1 \rangle} = T$
$x_{\langle 2 \rangle} = H$	0.5	0
$x_{\langle 2 \rangle} = T$	0	0.5

Relational Lifting

$$[x_{\langle 1 \rangle} = x_{\langle 2 \rangle}]$$

Probabilistic Relational Reasoning



handled by Probabilistic Relational Hoare Logic

Programs without Perfect Alignment

1

```
x :~ coin( );
```

```
y :~ coin( );
```

2

```
y :~ coin( );
```

```
x :~ coin( );
```

Need to go outside of the relational logic to derive

$$\llbracket \mathbf{x}\langle 1 \rangle = \mathbf{x}\langle 2 \rangle \wedge \mathbf{y}\langle 1 \rangle = \mathbf{y}\langle 2 \rangle \rrbracket$$

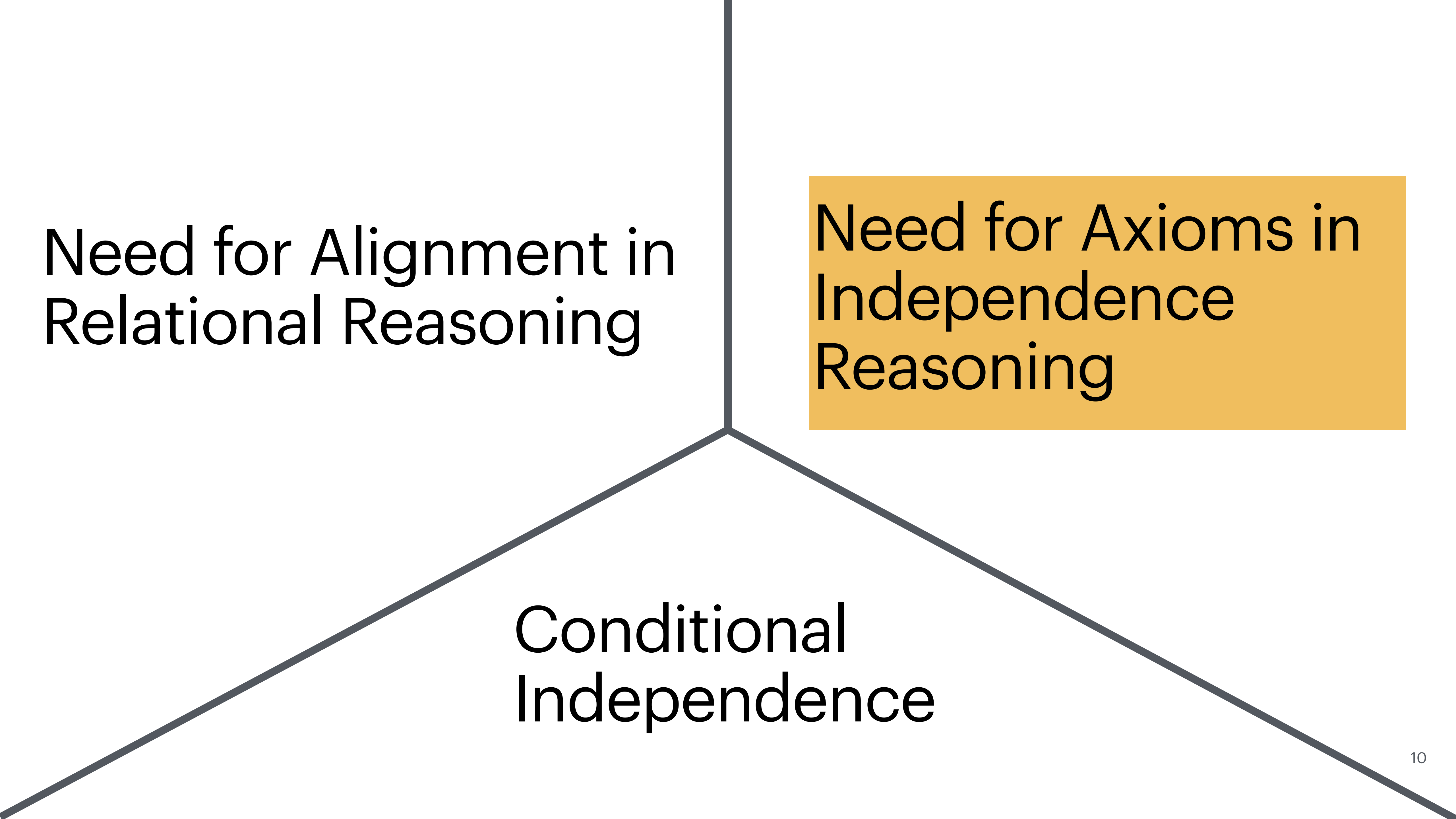
1

```
x :~ coin( );  
y :~ coin( );
```

2

```
y :~ coin( );  
x :~ coin( );
```

We want to prove **within the logic** that such simple swap does not change the output distribution.

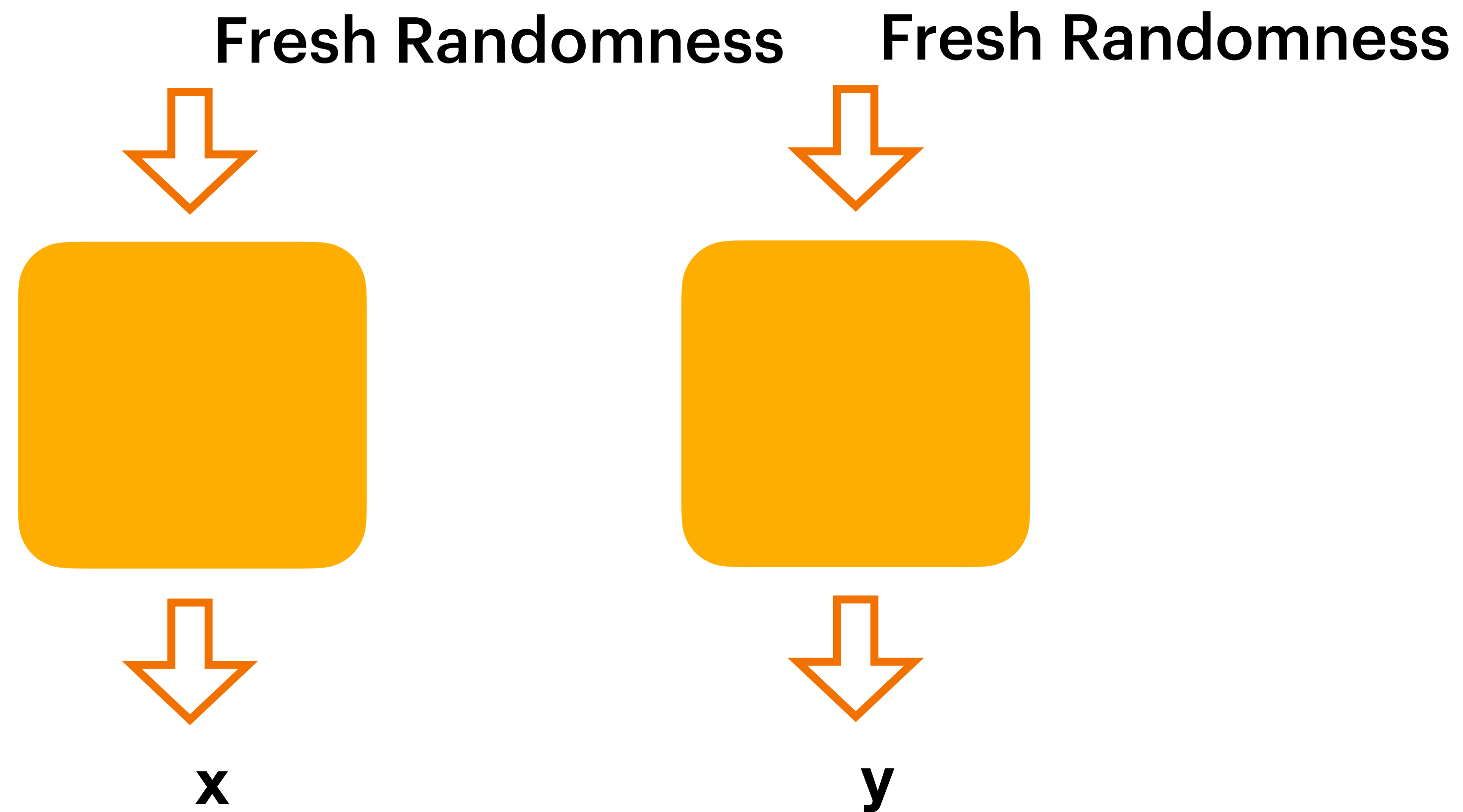


Need for Alignment in
Relational Reasoning

Need for Axioms in
Independence
Reasoning

Conditional
Independence

Limitations of Existing Probabilistic Separation Logic



A Simple Example that Causes Problem

`m := ~ coin( );`

`k := ~ coin( );`

`c := m xor k`

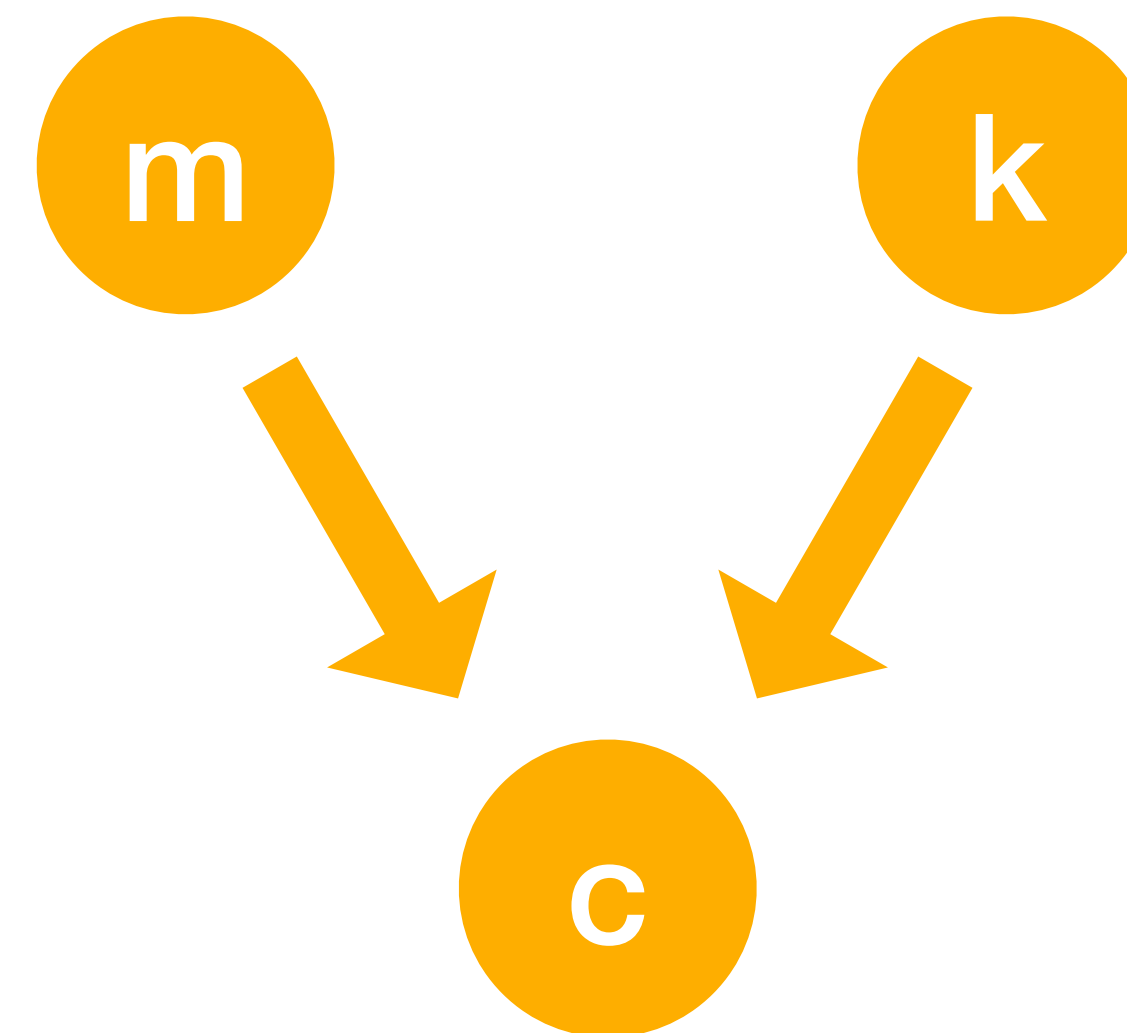
`m, k independent`

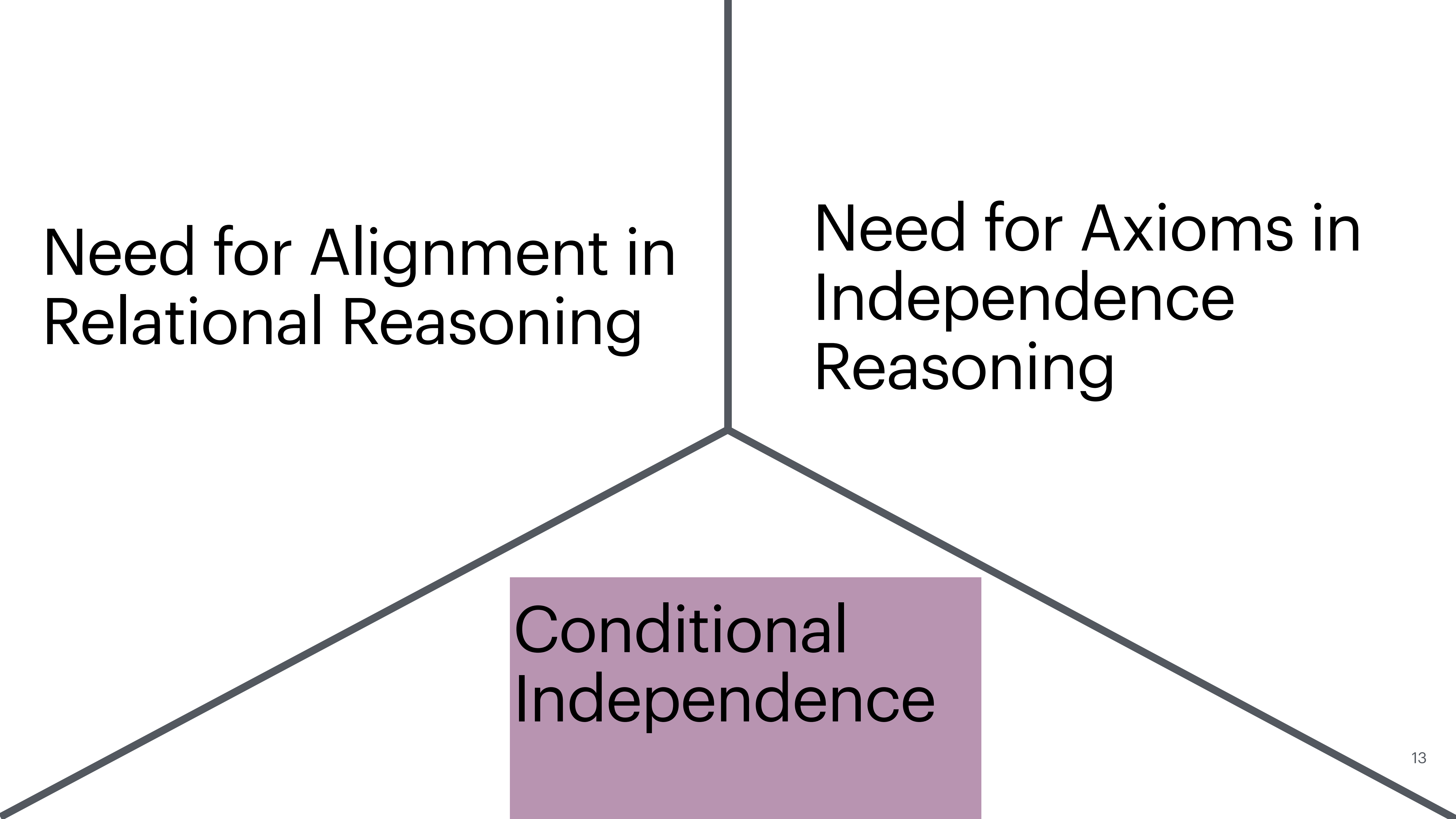


`m, c independent`



`k, c independent`





Need for Alignment in
Relational Reasoning

Need for Axioms in
Independence
Reasoning

Conditional
Independence

Sometimes, we do not have
independence and want to show
conditional independence

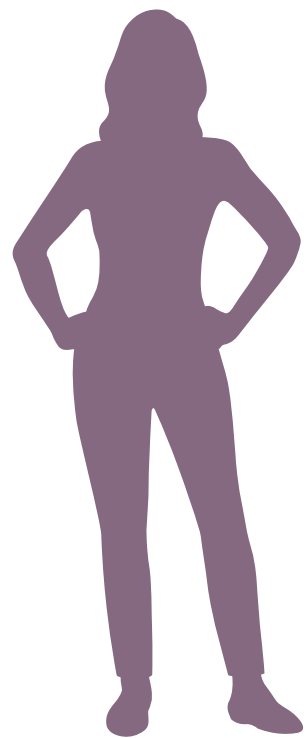
Multi-Party Secure Computation



Secret b

Compute $a + b + c \mod p$

Secret a



Secret c



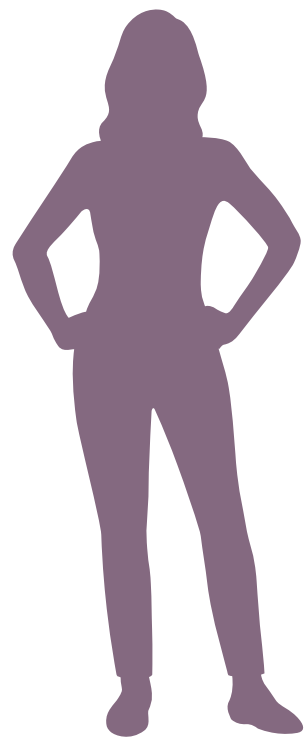
Multi-Party Secure Computation



Secret b

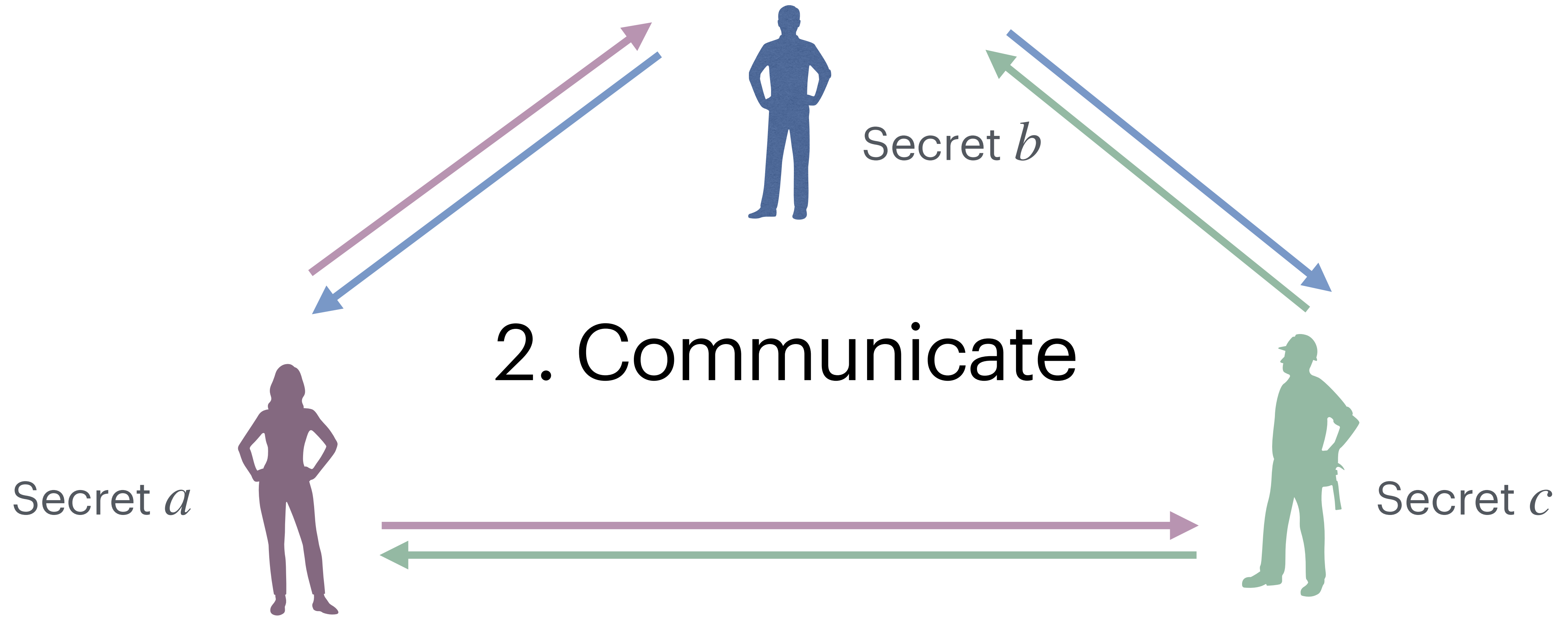
1. Each splits secret into 3 shares

Secret a



Secret c

Multi-Party Secure Computation



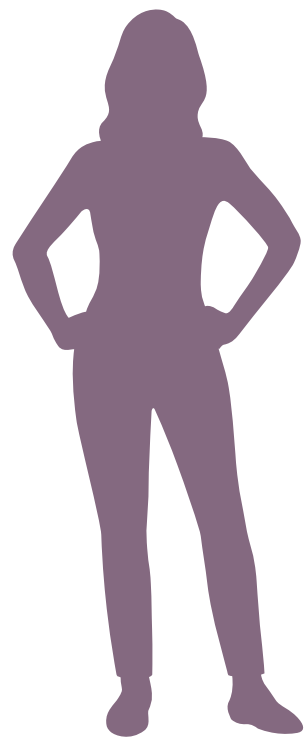
Multi-Party Secure Computation



Secret b

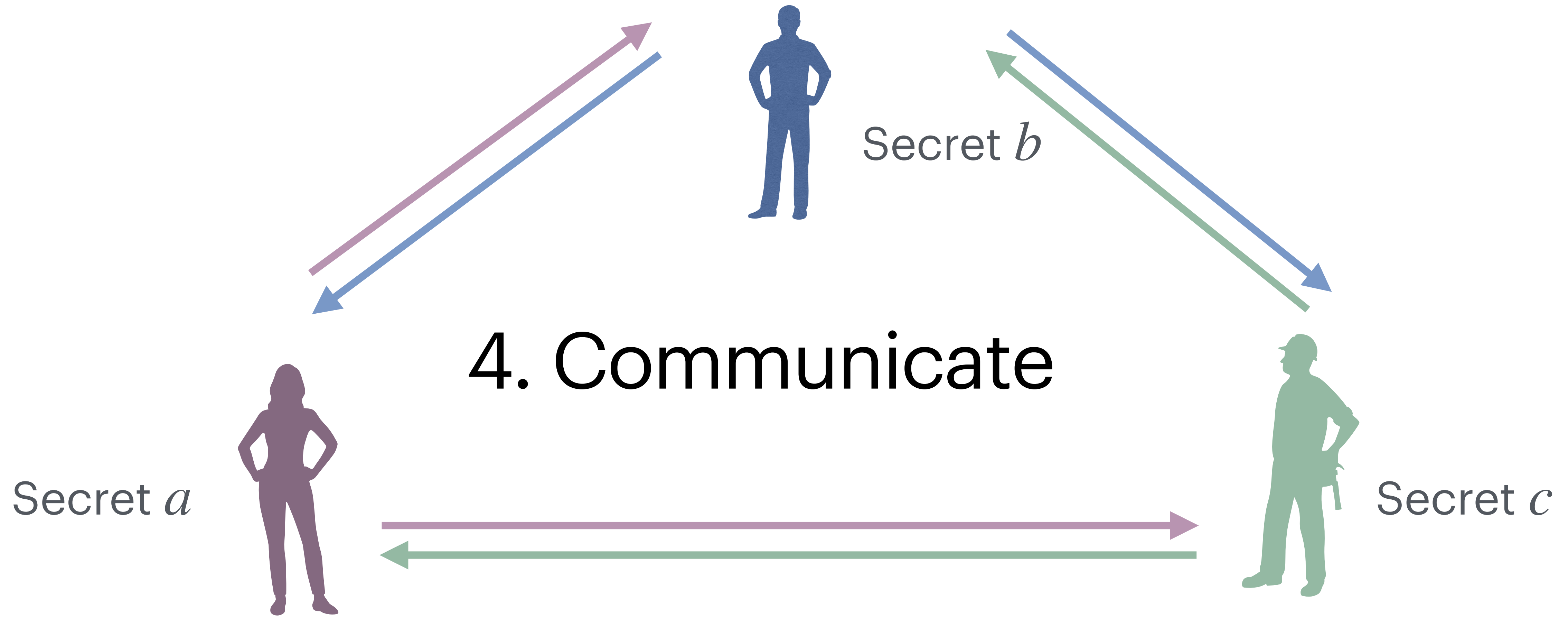
3. Compute

Secret a

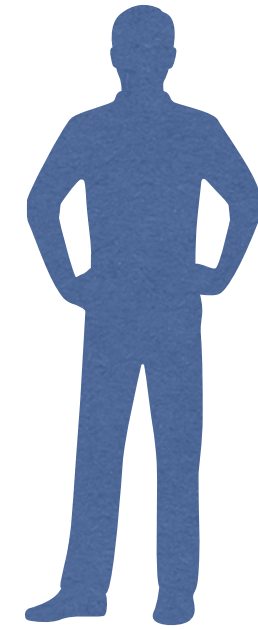


Secret c

Multi-Party Secure Computation



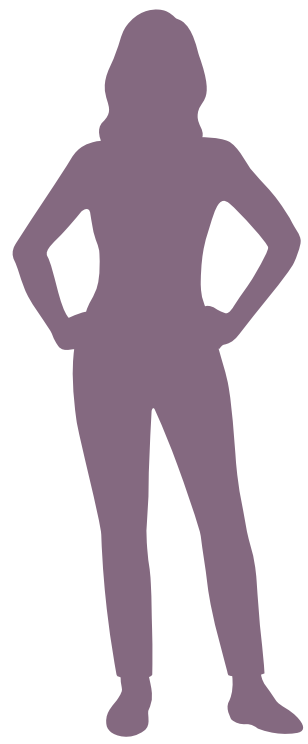
Multi-Party Secure Computation



Secret b

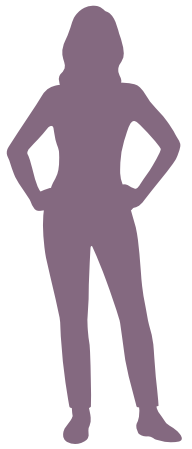
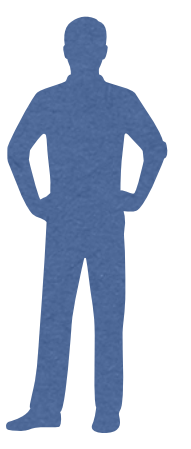
5. Compute the sum

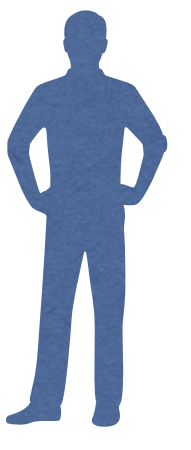
Secret a



Secret c

Perfect security for  :

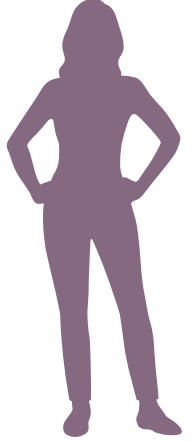
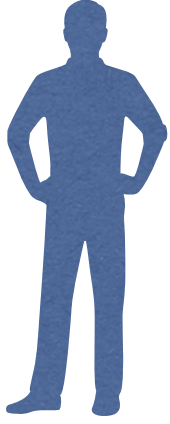
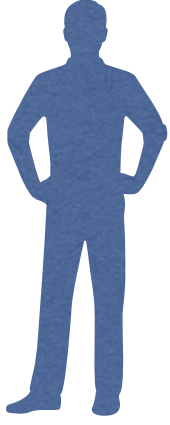
 's view is independent from  's secret, and

 's view is independent from  's secret.

But we do want to **reveal the sum** of three secrets, so we cannot keep each secrets perfectly secure.

What we want is that no more information other than the sum is revealed.

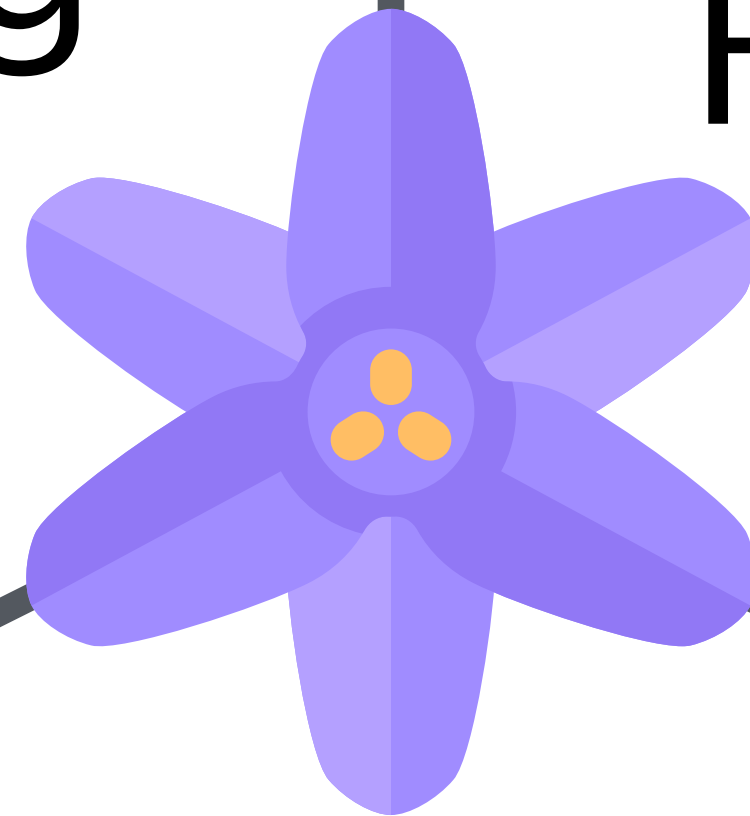
Condition on the sum of the secrets,

's view is independent from 's secret, and
's view is independent from 's secret.

Existing program logic cannot prove this conditional independence without additional axioms.

Need for Alignment in
Relational Reasoning

Need for Axioms in
Independence
Reasoning

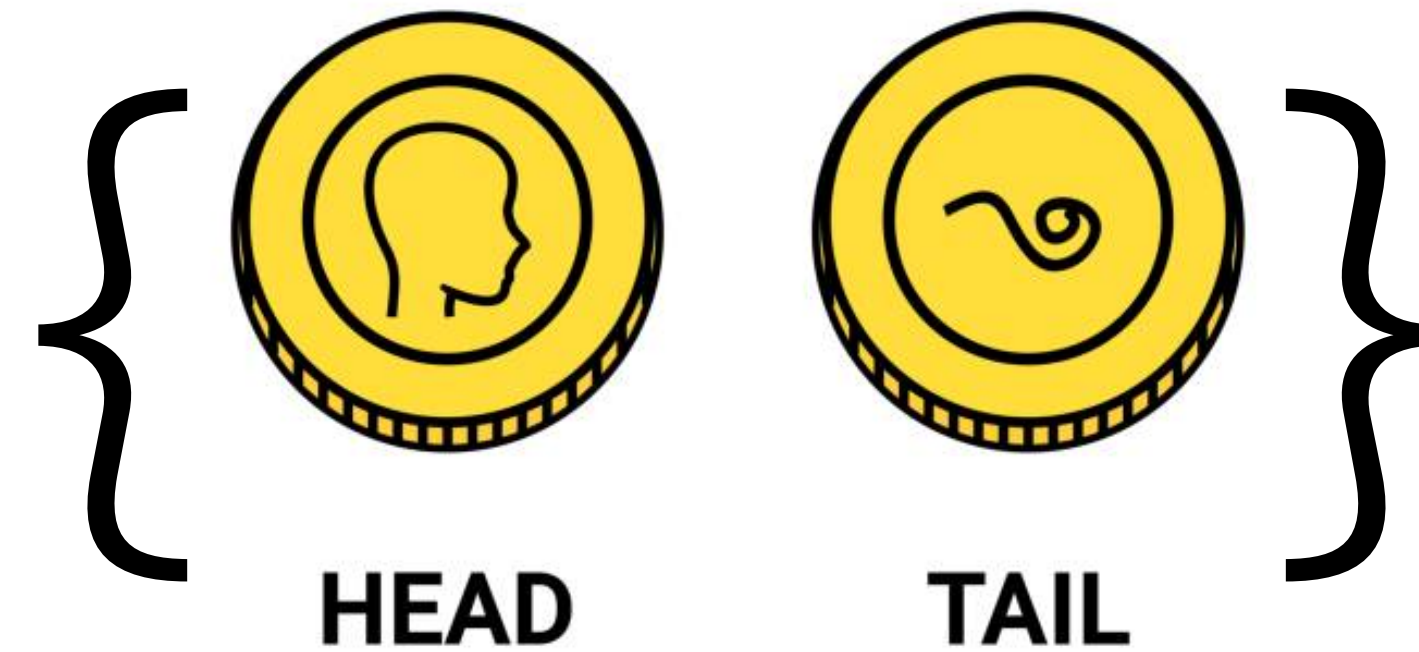


Conditional
Independence

Preliminaries

Probability Spaces

Outcome space Ω



Sigma algebra $\mathcal{F} \subseteq 2^\Omega$

Measure $\mu : \mathcal{F} \rightarrow [0,1]$

Programming Language (Unary)

$t ::= \text{skip}$

| $x := e$ (assignment)

| $x \sim d$ (sampling)

| $t1; t2$ (sequencing)

| $\text{if } e \text{ then } t1 \text{ else } t2$ (branching)

| $\text{repeat } e \text{ } t$ (bounded loop)

Program Semantics (Unary)

Stores: $\mathbb{S} = X \rightarrow V$

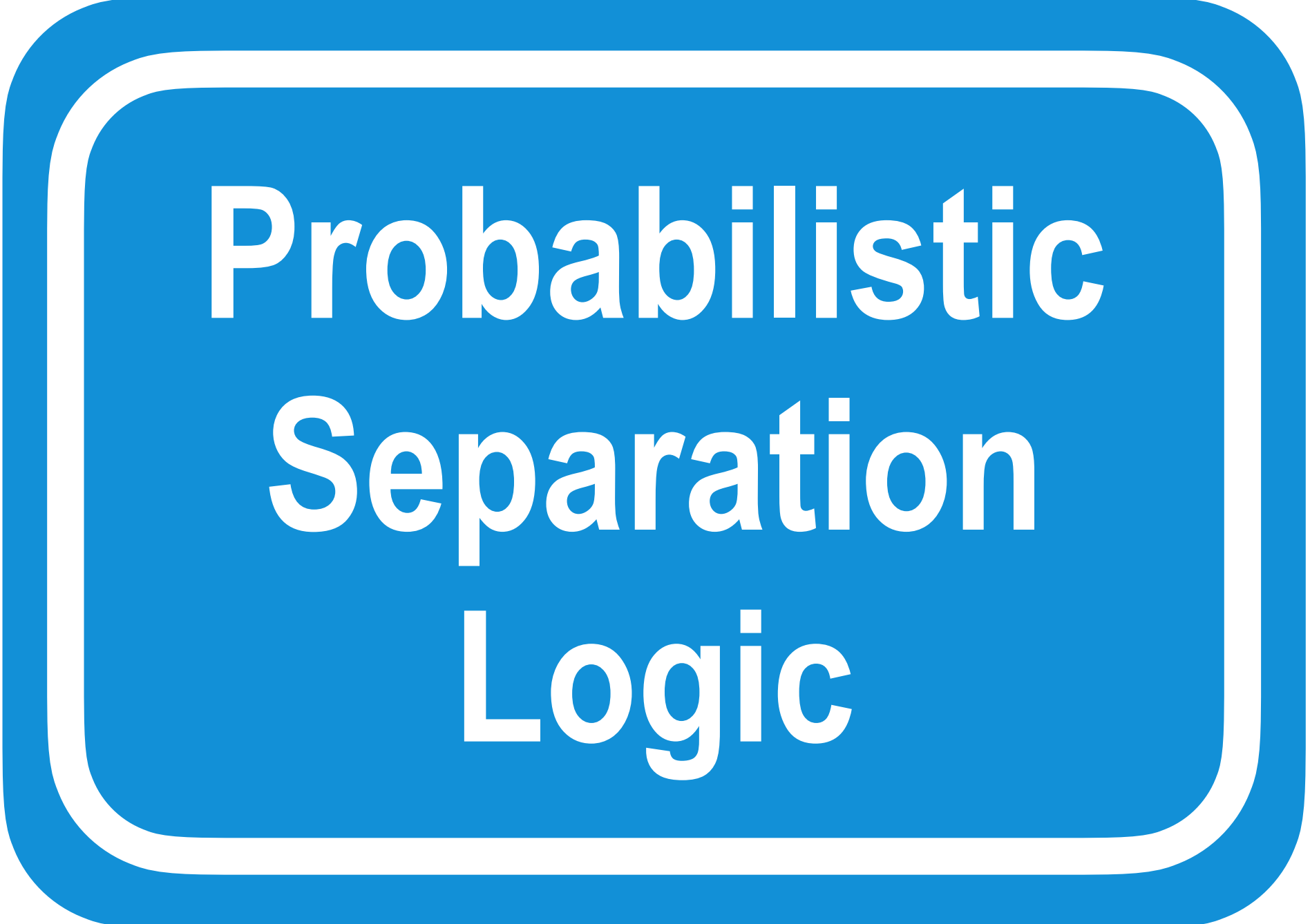
Semantics: $\mathbb{D}(\Sigma_{\mathbb{S}}) \rightarrow \mathbb{D}(\Sigma_{\mathbb{S}})$ where $\Sigma_{\mathbb{S}}$ is
the full sigma algebra on $\mathbb{S}$

Programming Language (Binary)

Binary Program [1: t1, 2:t2]

Semantics:

$$(\{0,1\} \rightarrow \mathbb{D}(\Sigma_{\mathcal{S}})) \rightarrow (\{0,1\} \rightarrow \mathbb{D}(\Sigma_{\mathcal{S}}))$$



Probabilistic Separation Logic

Assertions in Probabilistic Separation Logic

$P ::= \dots$ (Standard First-Order Formula)
 $| [e1 = e2]$ (Almost-sure Equality)
 $| e \sim \mu$ (Distributional Equality)

$x = H, y = H$	0.5
$x = H, y = T$	0
$x = T, y = H$	0
$x = T, y = T$	0.5

$$[x = y] \\ \wedge x \sim \text{Bern}(0.5) \wedge y \sim \text{Bern}(0.5)$$

$x = H, y = H$	0.25
$x = H, y = T$	0.25
$x = T, y = H$	0.25
$x = T, y = T$	0.25

$$x \sim \text{Bern}(0.5) \wedge y \sim \text{Bern}(0.5)$$

Assertions in Probabilistic Separation Logic

$P ::= \dots$	(Standard First-Order Formula)
$ [e1 = e2]$	(Almost-sure Equality)
$ e \sim \mu$	(Distributional Equality)
$ P1 * P2$	(Independence)

Independence Assertion

$(\mathcal{F}, \mu) \models P_1 * P_2$ iff there exists $(\mathcal{F}_1, \mu_1)$,
 $(\mathcal{F}_2, \mu_2)$ such that their **independent**
composition $(\mathcal{F}_1, \mu_1) \otimes (\mathcal{F}_2, \mu_2) \sqsubseteq (\mathcal{F}, \mu)$
and $(\mathcal{F}_1, \mu_1) \models P_1$, $(\mathcal{F}_2, \mu_2) \models P_2$

Independence Assertion

$$(\mathcal{F}, \mu) \models (\exists \mu_1 . x \sim \mu_1) * (\exists \mu_2 . y \sim \mu_2)$$

iff x, y are independent in $(\mathcal{F}, \mu)$

Example Rules in Probabilistic Separation Logic

Samp

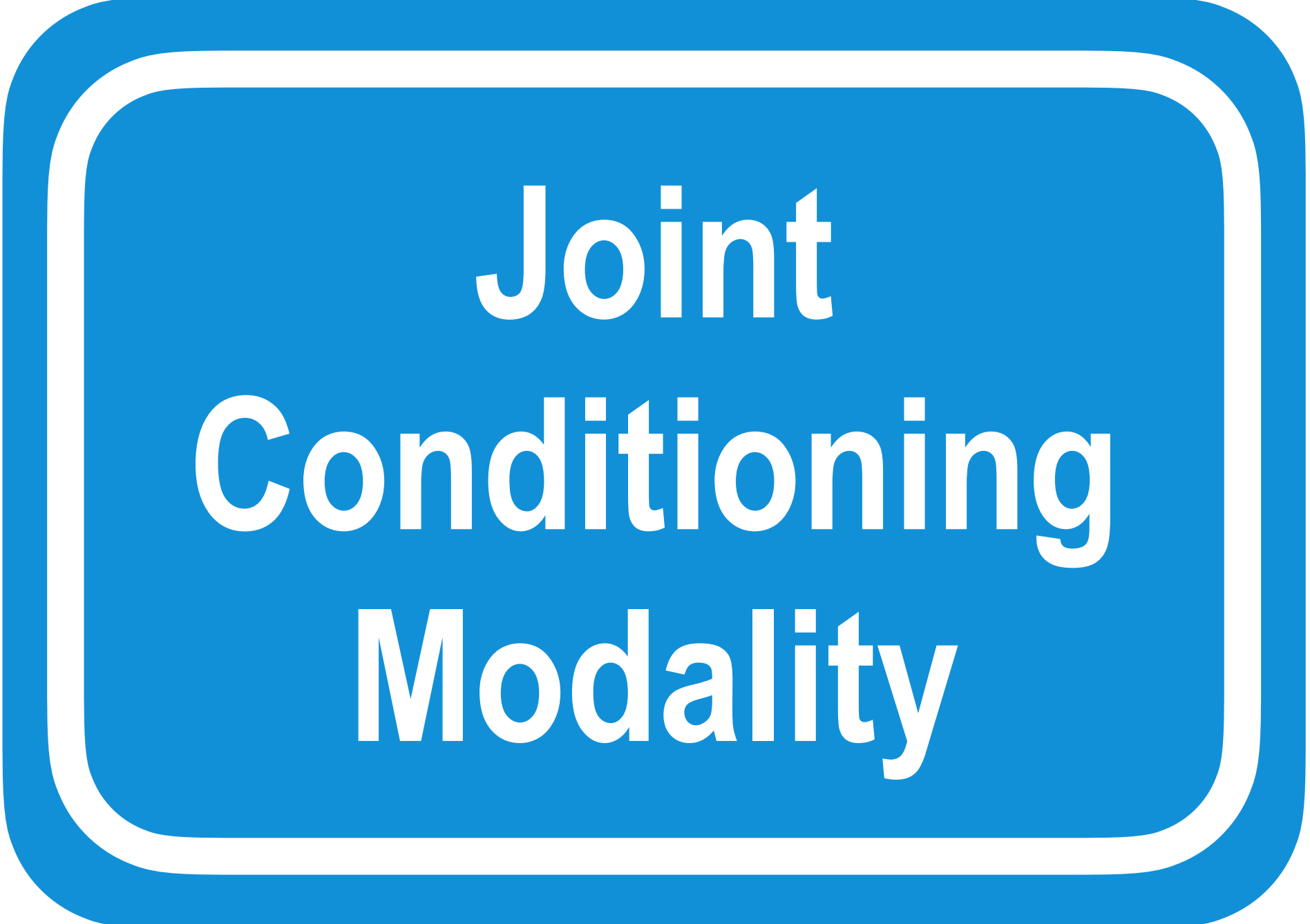
$$\frac{}{\vdash \{ \top \} x : \sim d \{ x \sim d \}}$$

Frame

$$\vdash \{ \phi \} c \{ \psi \}$$

optional side conditions

$$\vdash \{ \phi * \eta \} c \{ \psi * \eta \}$$



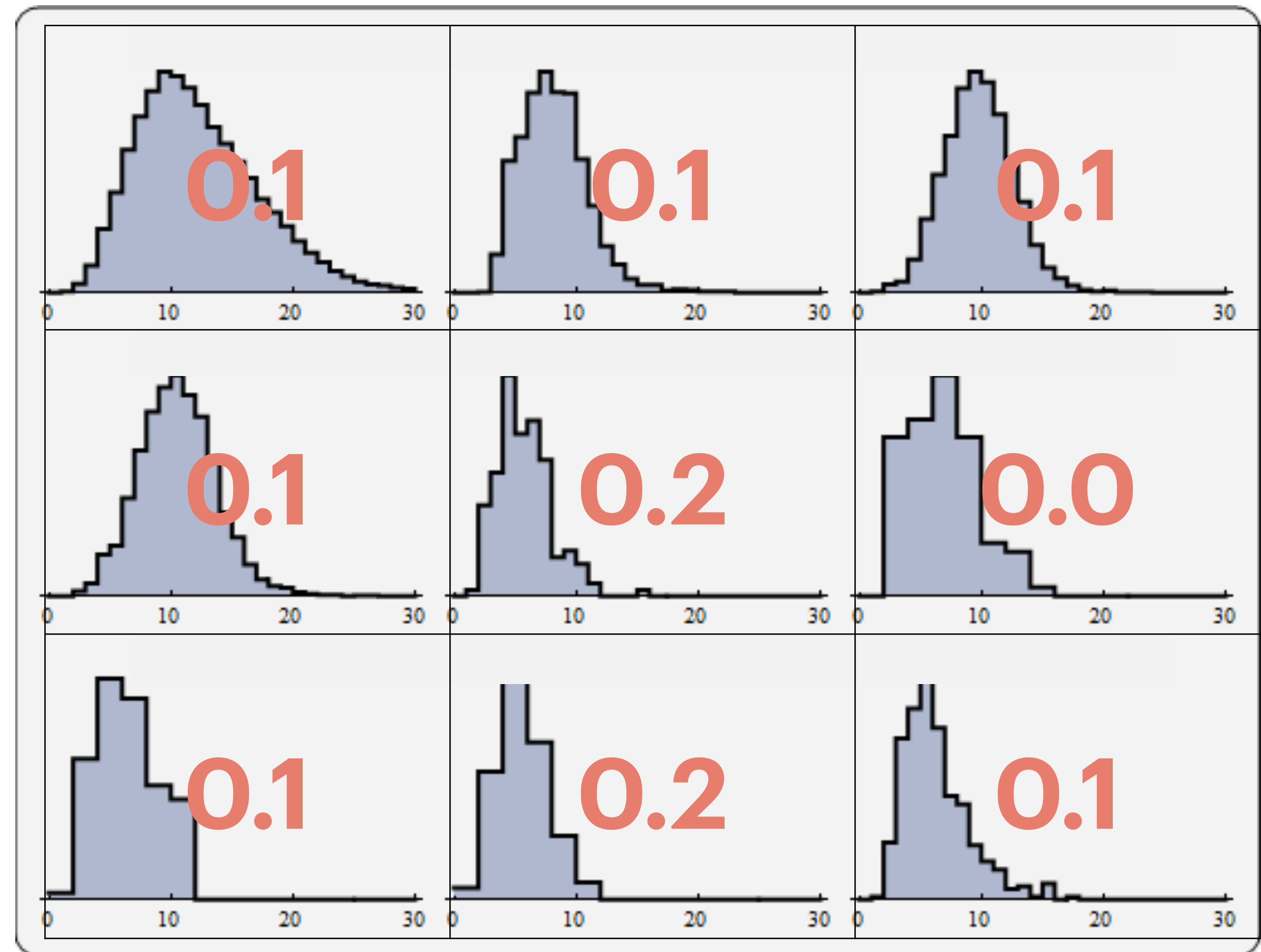
Joint Conditioning Modality

Binding Distribution to Markov Kernel

Distribution $\mu : \mathbb{D}(\Sigma_A)$

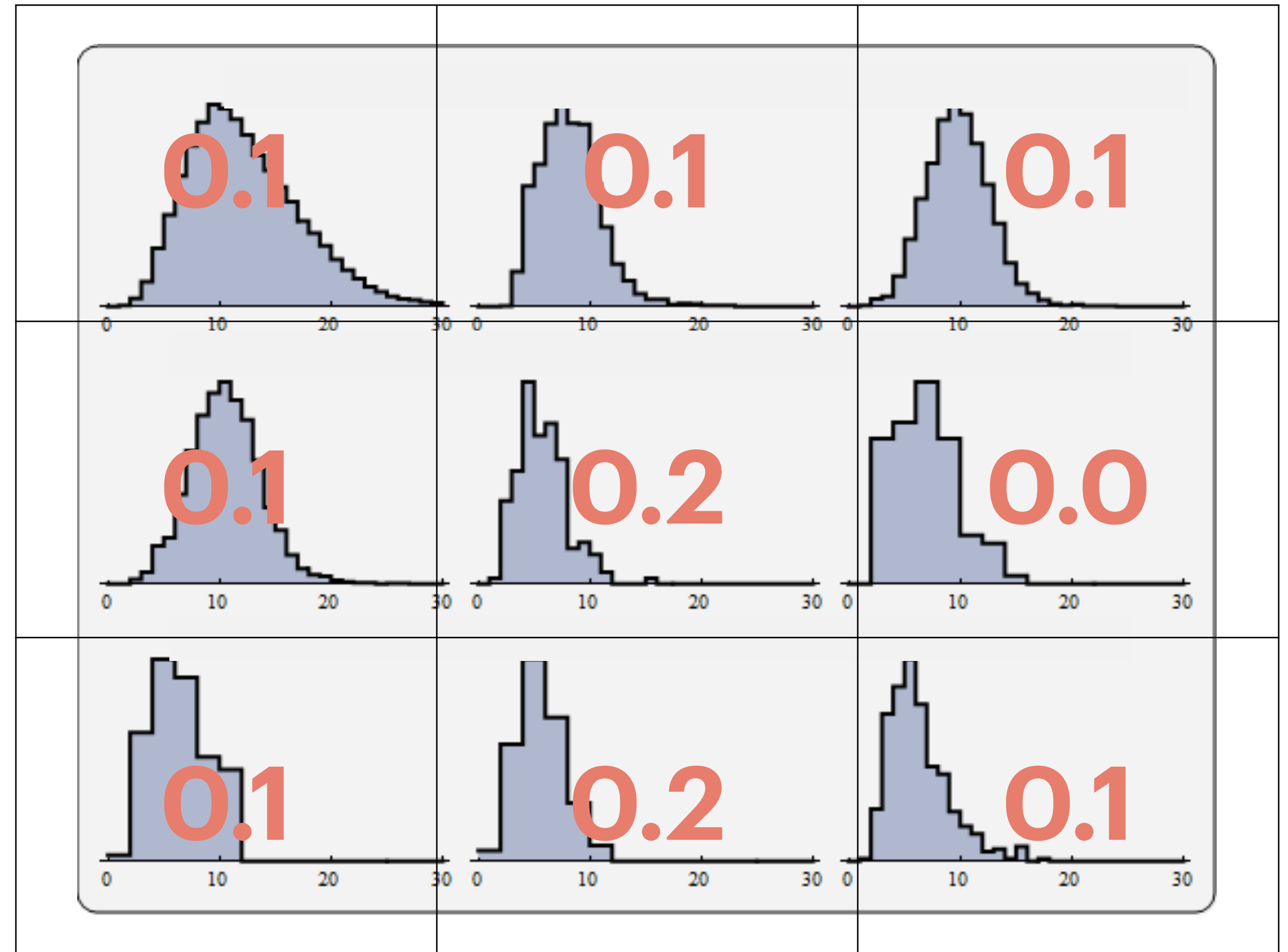
Kernel $\kappa : A \rightarrow \mathbb{D}(\Sigma_\Omega)$

$\mu \gg \kappa$



Conditioning Modality

$(\mathcal{F}, f) \models C_{\mu} v . P(v)$ if there exists κ such that $f = (\mu \gg \kappa)$ and for all value v in the support of μ , $(\mathcal{F}, \kappa(v)) \models P(v)$.



Example

$x = H, y = H$	0.5
$x = H, y = T$	0
$x = T, y = H$	0
$x = T, y = T$	0.5

f

$$(\mathcal{F}, f) \models C_f v . [x = y]$$

Joint Conditioning Modality

$1 : (\mathcal{F}_1, f_1), 2 : (\mathcal{F}_2, f_2) \models C_\mu v . P(v)$ if there exist kernels κ_1, κ_2 such that $f_1 = (\mu \gg= \kappa_1), ,$
 $f_2 = (\mu \gg= \kappa_2)$ and for all value v in the support of μ ,

$1 : (\mathcal{F}_1, \kappa_1(v)), 2 : (\mathcal{F}_2, \kappa_2(v)) \models P(v).$

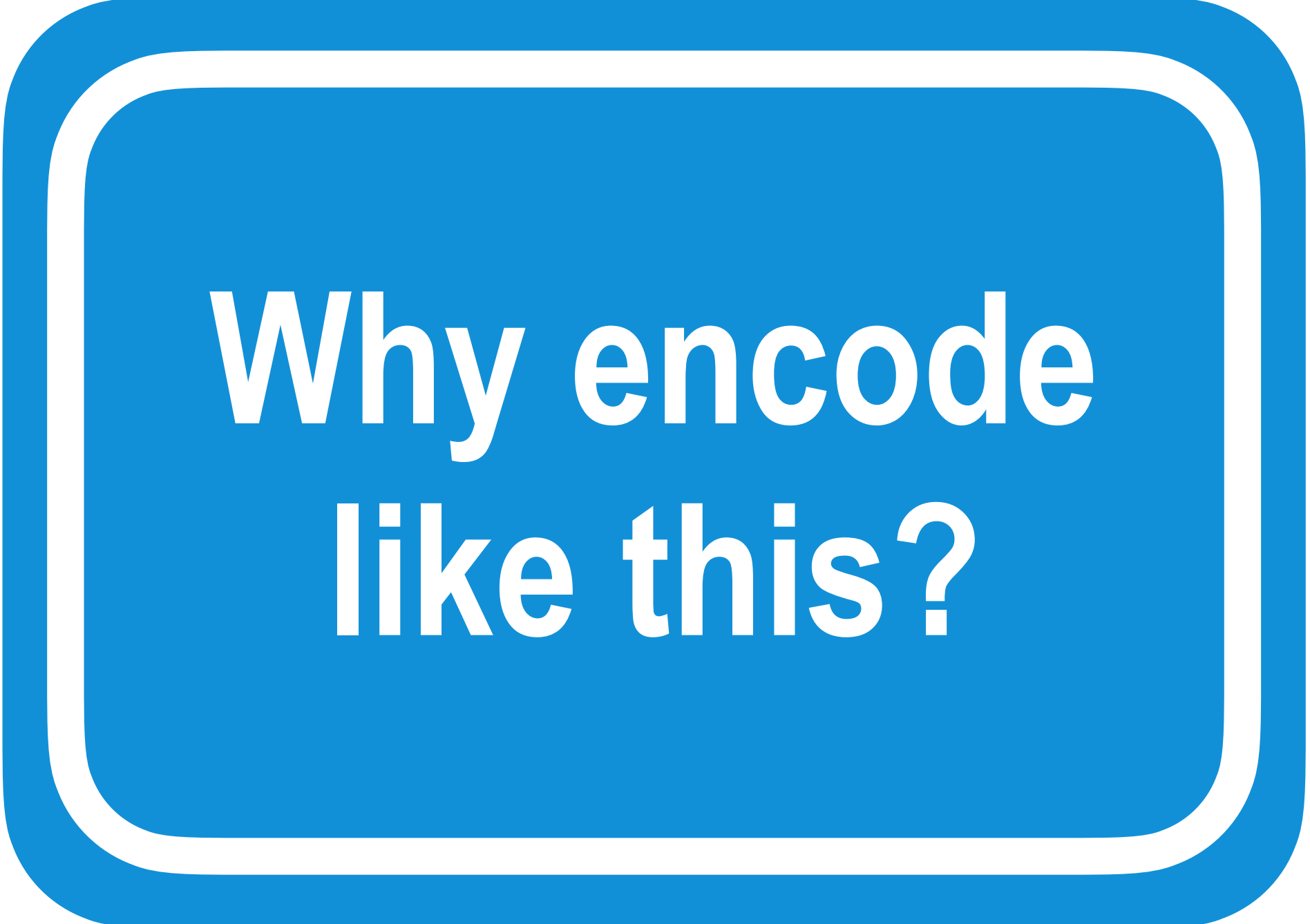
Insight 1: We can use joint conditioning to assert μ is a probabilistic coupling of f_1 and f_2

$$1 : (\mathcal{F}_1, f_1), 2 : (\mathcal{F}_2, f_2) \models C_\mu v . [X\langle 1 \rangle = v(X\langle 1 \rangle)] \wedge [Y\langle 2 \rangle = v(Y\langle 2 \rangle)]$$

Distributions f_1, f_2 satisfy relational lifting $[R]$ if there is a coupling μ of f_1, f_2 such that the support of μ satisfies R

Insight 2: Relational lifting is asserting a relation after conditioning on a coupling

$$[R] := \exists \mu . C_{\mu} v . [X\langle 1 \rangle = v(X\langle 1 \rangle)] \wedge [Y\langle 2 \rangle = v(Y\langle 2 \rangle)] \wedge R$$



**Why encode
like this?**

Benefit: We can derive a lot of relational reasoning principles from rules of conditioning modality

Rules for Conditioning Modality

It commutes with many connectives in the logic.

C-FOR-ALL

$$C_{\mu} v. \forall x : X. Q(v, x) \vdash \forall x : X. C_{\mu} v. Q(v, x)$$

Rules for Conditioning Modality

It inherits rules from monadic laws.

C-UNIT-L

$$\mathbf{C}_{\delta_{v_0}} v. K(v) \dashv\vdash K(v_0)$$

C-UNIT-R

$$E\langle i \rangle \sim \mu \dashv\vdash \mathbf{C}_\mu v. \lceil E\langle i \rangle = v \rceil$$

C-ASSOC

$$\frac{\mu_0 = \mathbf{bind}(\mu, \lambda v. (\mathbf{bind}(\kappa(v), \lambda w. \mathbf{return}(v, w))))}{\mathbf{C}_\mu v. \mathbf{C}_{\kappa(v)} w. K(v, w) \vdash \mathbf{C}_{\mu_0}(v, w). K(v, w)}$$

Rules for Conditioning Modality

It also has other interesting rules. E.g.,

C-TRANSF

$$\frac{\begin{array}{l} f: \text{supp}(\mu') \rightarrow \text{supp}(\mu) \text{ bijective} \\ \forall b \in \text{supp}(\mu'). \mu'(b) = \mu(f(b)) \end{array}}{C_{\mu} a. K(a) \vdash C_{\mu'} b. K(f(b))}$$

From this rules for C_μ and our program logic rules, we can derive

- All rules in Probabilistic Relational Hoare Logic (pRHL)

- New principles, e.g., **RL-MERGE**
$$\llbracket R_1 \rrbracket * \llbracket R_2 \rrbracket \vdash \llbracket R_1 \wedge R_2 \rrbracket$$

We can now prove inside the logic that

1

```
x :~ coin( );
```

```
y :~ coin( );
```

2

```
y :~ coin( );
```

```
x :~ coin( );
```

$$\llbracket \mathbf{x}\langle 1 \rangle = \mathbf{x}\langle 2 \rangle \wedge \mathbf{y}\langle 1 \rangle = \mathbf{y}\langle 2 \rangle \rrbracket$$

Other Usage of C Modality

One-time Pad Encryption

$m \sim \text{coin}()$

$k \sim \text{coin}()$

$c = m \text{ xor } k$

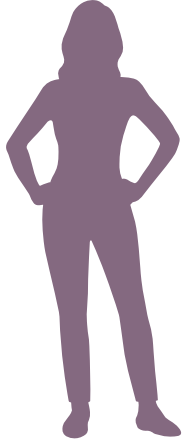
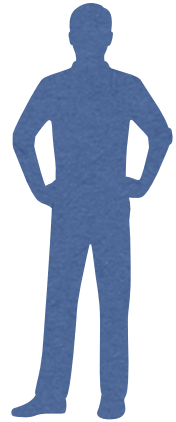
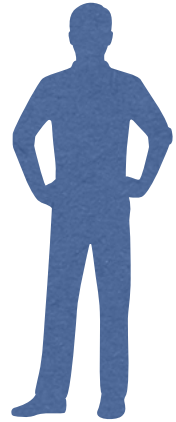
$\{m \sim \text{Bern}(0.5) * k \sim \text{Bern}(0.5) * [c = m \text{ xor } k]\}$

After applying several rules,

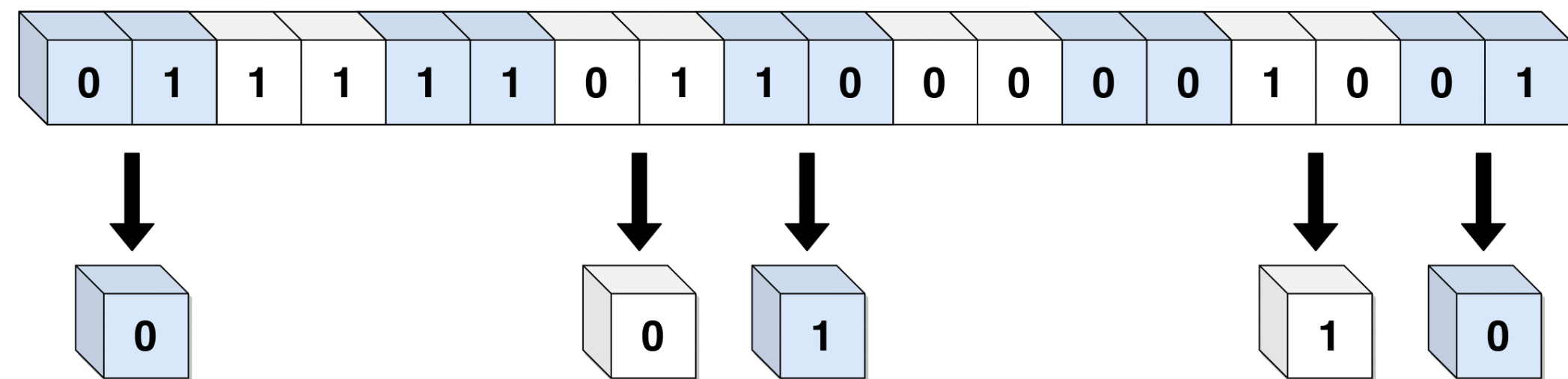
$\{m \sim \text{Bern}(0.5) * c \sim \text{Bern}(0.5)\}$

Expressing and Proving Conditional Independence

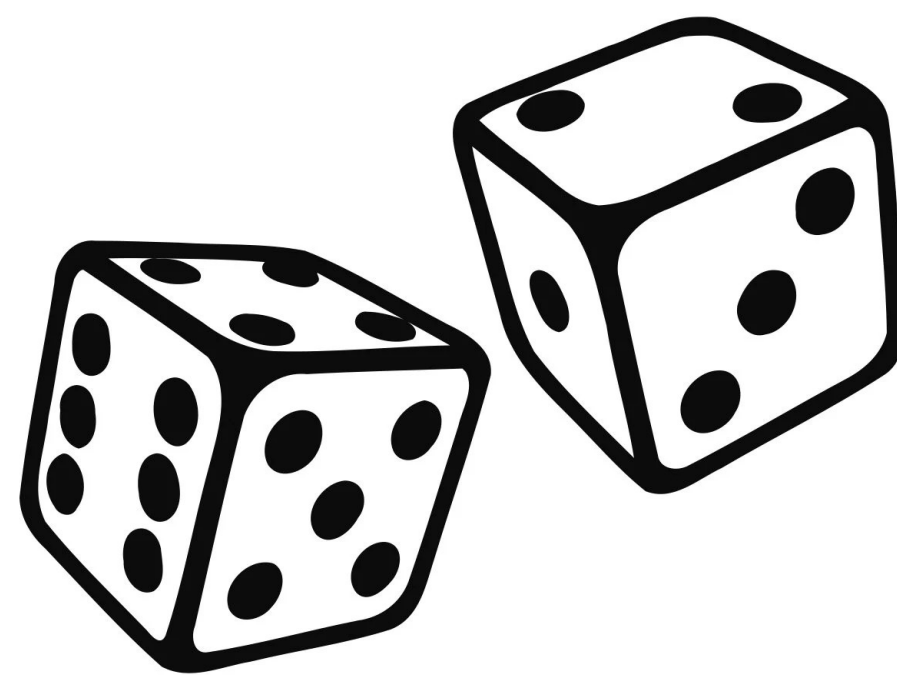
Condition on the sum of the secrets,

 's view is independent from  's secret, and
 's view is independent from  's secret.

More Examples



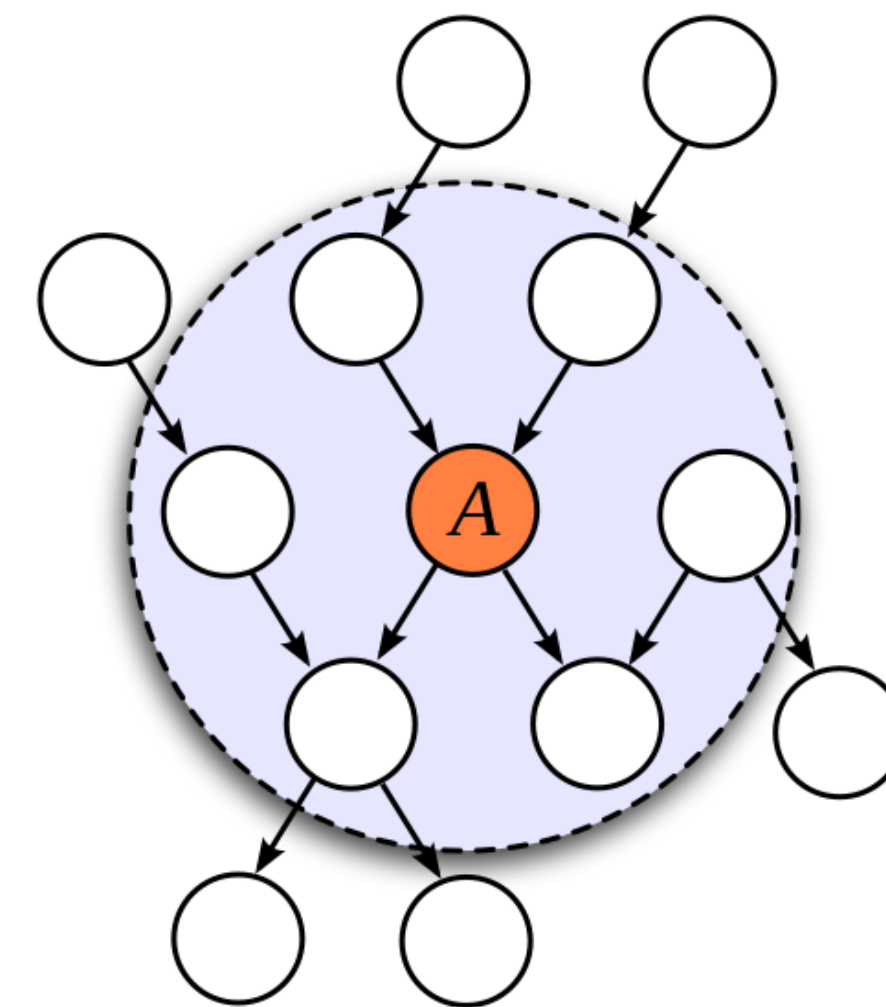
Von Neumann Extractor



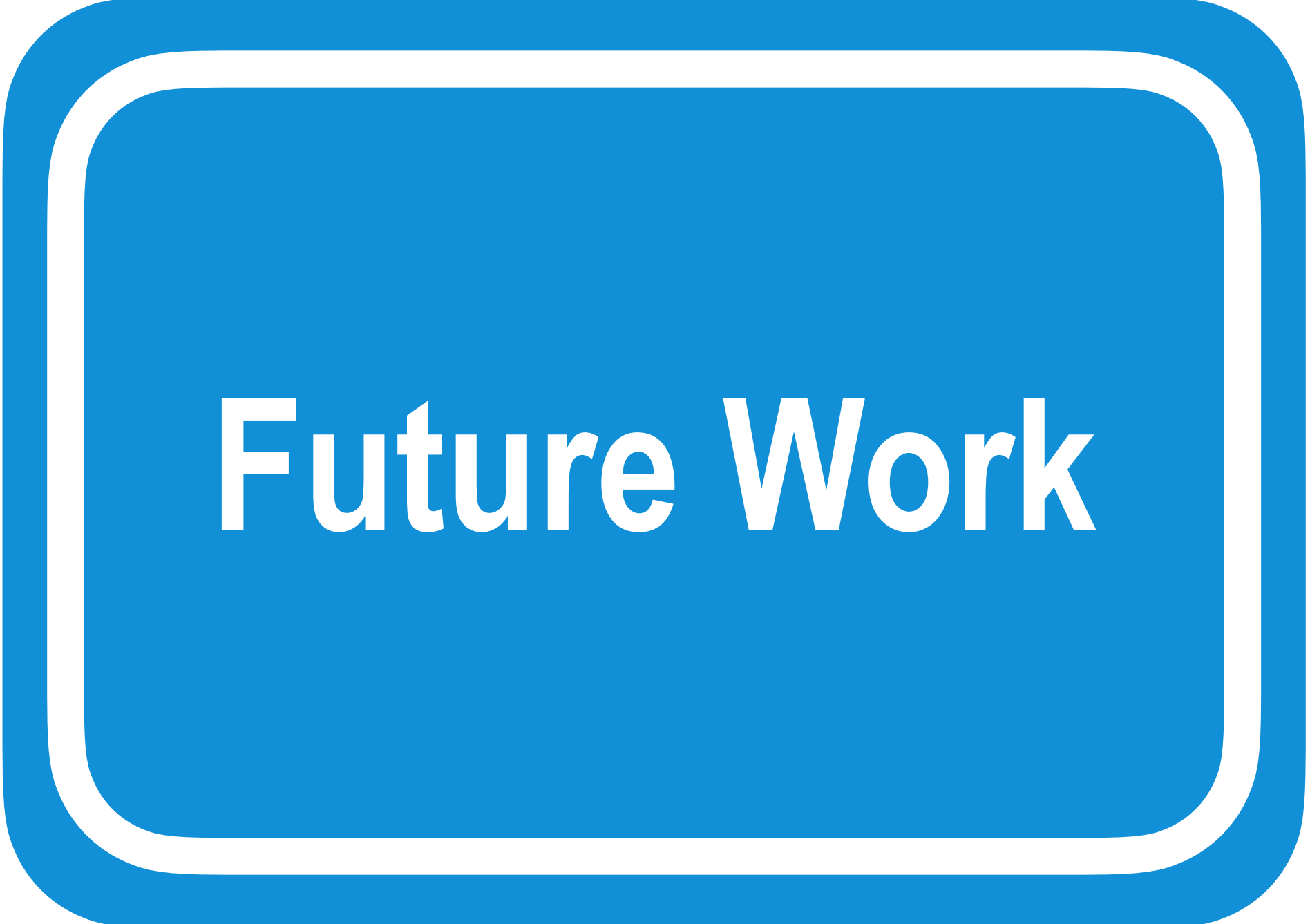
Monte Carlo Estimation



Reservoir Sampling



Markov Blanket



Future Work

- A loop rule for unbounded loops
- Automation of the logic
- Supports for continuous probability
- Language level supports for distributed computations
- ...



Questions